# RISC-V Processor (RV64I Subset)

## Report

**Course:** Introduction to Processor Architecture (IPA)

**Assignment:** Pipelined-Processor-Project

---


### Team Members

| | |
|---|---|
| **Jayakanth M V** | 2025122004 |
| **Lavanya Patil** | 2024112016 |
| **Nihaal Appana** | 2025122007 |


---

# Contents

**1 Pipeline Implementation** **2**
1.1 About Pipelining . . . . . 2
1.2 Introduction to Pipelining and Pipeline Registers . . . . . 3
1.3 Pipelining Flow and Stages . . . . . 4
1.4 Forwarding and Register Data Passing . . . . . 4
1.5 Final Pipelined RISC-V Diagram . . . . . 6
1.6 Plots & Screenshots . . . . . 10
1.7 Conclusion . . . . . 13

**Contributions** **15**

# 5-Stage Pipeline RISC-V

## 1 Pipeline Implementation

### 1.1 About Pipelining

Pipelining is a processor implementation technique in which multiple instructions are **overlapped** during execution. Rather than waiting for one instruction to complete all of its stages before the next instruction begins, a pipelined processor allows successive instructions to enter the datapath one after another, each occupying a different stage at any given clock cycle. As Harris & Harris describe in *Digital Design and Computer Architecture*, pipelining is analogous to an assembly line: just as a manufacturing plant increases throughput by working on multiple products at different stages of production simultaneously, a processor increases throughput by working on multiple instructions at different pipeline stages simultaneously [1].

In a traditional **single-cycle (sequential)** design, every instruction must traverse the entire datapath - fetch, decode, execute, memory access, and write-back - within a single clock cycle. The clock period is therefore dictated by the *slowest* instruction, and all functional units sit idle for the portion of the cycle they are not needed. This leads to poor **throughput**: only one instruction completes per (long) cycle.

Pipelining addresses this inefficiency by dividing execution into discrete stages and allowing each stage to work on a *different* instruction simultaneously. Although the **latency** of a single instruction does not decrease - it still passes through all stages - the **throughput** increases dramatically. In the ideal case, once the pipeline is filled, one instruction completes every clock cycle.

**Performance trade-off.** The maximum speedup achievable by an $N$-stage pipeline is $N$. In practice, two factors reduce this: (1) pipeline stages are not perfectly balanced, so the clock period is set by the *slowest* stage, and (2) *hazards* force the pipeline to stall or flush, wasting cycles.

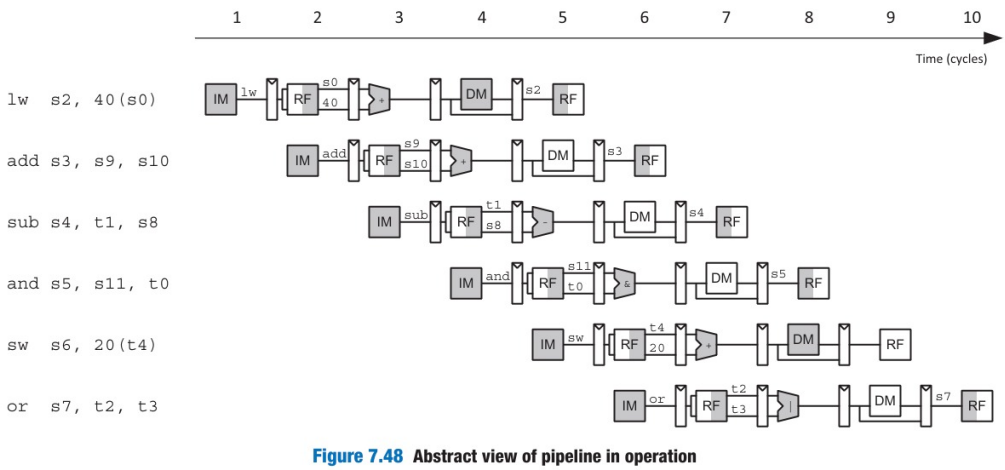


Figure 1: Pipelined Execution: multiple instructions occupy different stages simultaneously, maximising hardware utilisation.

## 1.2 Introduction to Pipelining and Pipeline Registers

To correctly overlap instructions, intermediate results produced by one stage must be preserved and passed to the next stage on every clock edge. This is accomplished by inserting **pipeline registers** between consecutive stages - banks of flip-flops that capture all data and control signals produced by the preceding stage and hold them stable for the following stage.

A critical design principle highlighted by Harris & Harris is that *all signals associated with a particular instruction must advance through the pipeline in unison* [1]. Failing to propagate a signal (e.g., the destination register address `rd`) through every stage causes subtle but fatal mismatches - for example, a result being written to the wrong register in the Writeback stage. Every control signal generated in the Decode stage must therefore be pipelined alongside the data it controls.

The four pipeline registers used in the classic five-stage RISC-V pipeline are:

1. **IF/ID** (Instruction Fetch / Instruction Decode) - Stores the fetched instruction word and PC+4 so that the Decode stage can extract register addresses and immediate fields.
2. **ID/EX** (Instruction Decode / Execute) - Stores register data (`read_data1`, `read_data2`), the sign-extended immediate, the destination register address (`rd`), and *all control signals* needed by the Execute and later stages.
3. **EX/MEM** (Execute / Memory Access) - Stores the ALU result, branch target address, zero flag, write data, destination register address, and remaining control signals (`MemWrite`, `RegWrite`, `ResultSrc`).
4. **MEM/WB** (Memory Access / Write-Back) - Stores the memory read data or ALU result, the destination register address, and write-back control signals (`RegWrite`, `ResultSrc`).

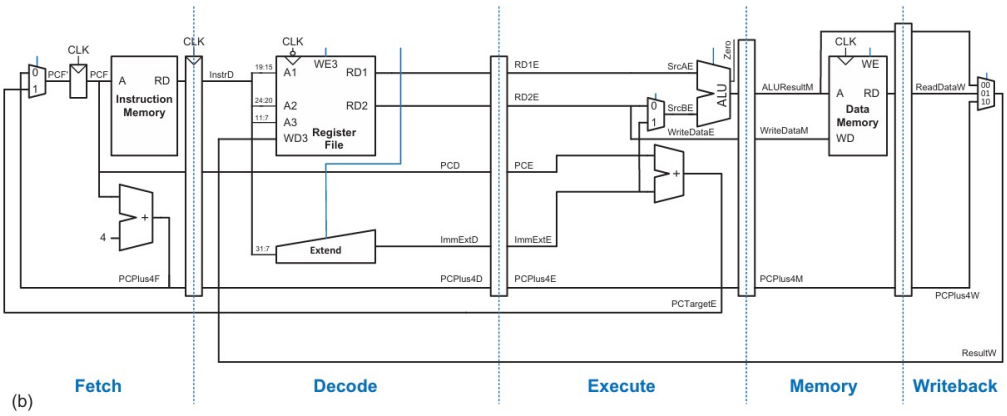


Figure 2: Pipeline Registers Between the Five Stages. All data *and* control signals must advance together through each register.

## 1.3 Pipelining Flow and Stages

The RISC-V pipeline is divided into five stages. In steady state, each stage processes a different instruction every clock cycle:

1. **IF - Instruction Fetch**: The instruction at the PC address is read from Instruction Memory. The PC is incremented by 4. Both the instruction and PC+4 are written into the **IF/ID** register.

2. **ID - Instruction Decode / Register Read**: The opcode is decoded by the Control Unit, source registers (`rs1`, `rs2`) are read, and the immediate is sign-extended. All outputs are latched into **ID/EX**.

   *An important subtlety noted by Harris & Harris: the register file is written in the **first half** of a clock cycle and read in the **second half**. A value written by an instruction in Writeback can therefore be read by an instruction in Decode within the same cycle, eliminating one class of hazard without forwarding [1].*

3. **EX - Execute / Address Calculation**: The ALU performs the required operation. The branch target address is computed by adding the immediate to the PC. Results are stored in **EX/MEM**.

4. **MEM - Memory Access**: Load instructions read Data Memory; store instructions write it. ALU instructions pass their result through. Results are latched into **MEM/WB**.

5. **WB - Write-Back**: The result (from memory or ALU) is selected by the `ResultSrc` multiplexer and written back to the destination register, provided `RegWrite` is asserted.

Because each instruction occupies a different stage on every cycle, up to **five instructions** can be in flight simultaneously.

## 1.4 Forwarding and Register Data Passing

**Why forwarding is necessary.** A **data hazard** arises when an instruction in the Execute stage reads a register that a preceding instruction still in the pipeline has not yet

written back. Harris & Harris classify these as **Read After Write (RAW) hazards** [1]: the dependent instruction reads a register before a prior instruction has written it, causing a **stale value** to be used.

To understand the exact hazard window, consider an instruction that writes register `x1`:

- The instruction immediately following reads `x1` during Decode while the producer is in Execute - its result is not yet available. RAW hazard.
- The instruction two positions later reads `x1` while the producer is in Memory - still not written back. RAW hazard.
- The instruction three positions later reads `x1` in the *second half* of Decode while the producer is in the *first half* of Writeback. Since the register file is written before it is read within the same cycle, this instruction obtains the **correct** value with no intervention required [1].

*Forwarding* resolves RAW hazards without stalling by routing the result directly from EX/MEM or MEM/WB back to the ALU inputs, bypassing the register file.

**Forwarding multiplexers and the Forwarding Unit.** Two multiplexers are placed in front of the ALU in the Execute stage. A **Forwarding Unit** compares the destination register (`rd`) of instructions in the Memory and Writeback stages against the source registers of the instruction in Execute and drives the multiplexer select lines. The complete forwarding conditions - along with the load-use stall and branch flush equations - are shown in Figure 7, reproduced directly from Harris & Harris [1].

The MEM-stage forward (`ForwardAE = 10`) takes priority because it carries the most recent result; the WB-stage forward (`ForwardAE = 01`) is used only if no MEM-stage conflict exists. The guard ($Rs1E \neq 0$) ensures `x0` - hardwired to zero in RISC-V - is never forwarded over. An analogous set of conditions determines `ForwardBE` for source operand `rs2`.

**Why data must propagate through every pipeline register.** Even though a result is first produced in Execute, it must continue through EX/MEM and MEM/WB for two reasons:

- Instructions further back in the pipeline may depend on the same result and will need it at the forwarding multiplexer inputs in later cycles.
- The Writeback stage must receive the data to update the register file; without propagation through MEM/WB the register file write never occurs.

**Why the destination register address must also be forwarded.** The Forwarding Unit performs register-*address* comparisons, so `rd` must be carried through every pipeline register alongside the data. Without it, the unit cannot detect whether a dependency exists [1].

## 1.5 Final Pipelined RISC-V Diagram

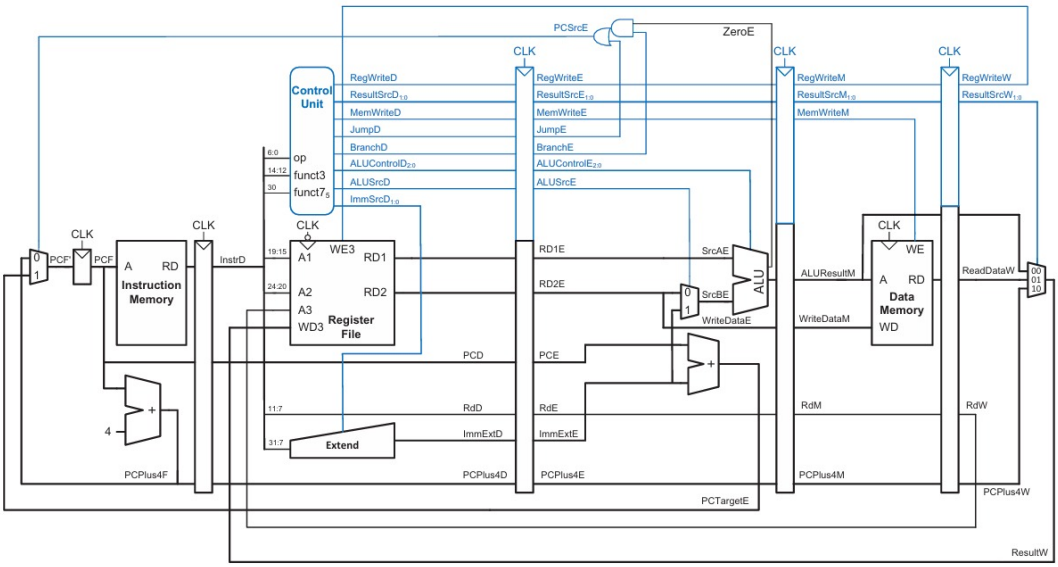


**Figure 7.51** Pipelined processor with control

Figure 3: Final Pipelined RISC-V Diagram.

## Hazards in Pipelining

A **hazard** is any situation that prevents the next instruction from executing in the immediately following cycle without producing an incorrect result. Harris & Harris identify two families [1]:

- **Data hazards** - an instruction reads a register not yet written back by a prior instruction (RAW dependency).
- **Control hazards** - the processor does not yet know which instruction to fetch next because a branch has not been resolved.

The complete set of hardware conditions used to detect and resolve all three hazard types is captured in Figure 7

## Types of Hazards and Their Solutions

### (a) Data Hazards - Forwarding (Bypassing)

**Solution - Forwarding (Bypassing):** The result is forwarded directly from EX/MEM or MEM/WB to the ALU input of the consuming instruction, eliminating stalls for back-to-back dependent instructions. Two forwarding paths are implemented:

- **EX/MEM → EX (MEM-stage forward):** The ALU result in EX/MEM is forwarded when the producing instruction is one cycle ahead of the consuming instruction.

- **MEM/WB → EX (WB-stage forward):** The result in MEM/WB is forwarded when the producing instruction is two cycles ahead, and no MEM-stage conflict exists for that operand.

The exact forwarding conditions are given in Figure 7.

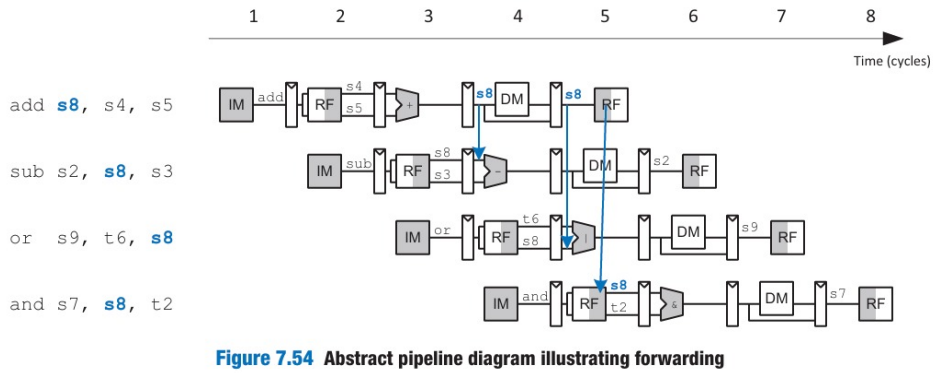


Figure 4: EX/MEM and MEM/WB Forwarding Paths Resolving a Data Hazard.

### (b) Load-Use Hazard - Stalling

Forwarding cannot resolve every hazard. With **load instructions** (`ld`), the loaded data is not available until the *end* of the Memory stage, yet a dependent instruction in Execute needs it at the *beginning* of that same cycle - data cannot be forwarded backwards in time.

**Example:**

```
ld   x1, 0(x2)   # data available at end of MEM (cycle 4)
add  x3, x1, x4  # needs x1 at start of EX    (cycle 4) -- too early!
```

**Solution - Pipeline Stall (Bubble):** The **Hazard Detection Unit** checks whether the instruction in Execute is a load (`ResultSrcE0 = 1`) and whether its destination (`RdE`) matches either source register of the instruction in Decode. The stall conditions are given in Figure 7. When triggered, the unit:

1. **Stalls** the IF and ID stages by disabling the enable inputs (EN) of their pipeline registers, holding current values.
2. Inserts a **NOP bubble** into Execute by asserting the synchronous clear (CLR) of ID/EX, zeroing all control signals [1].

After the one-cycle stall the loaded value is available in MEM/WB and forwarded normally to Execute.

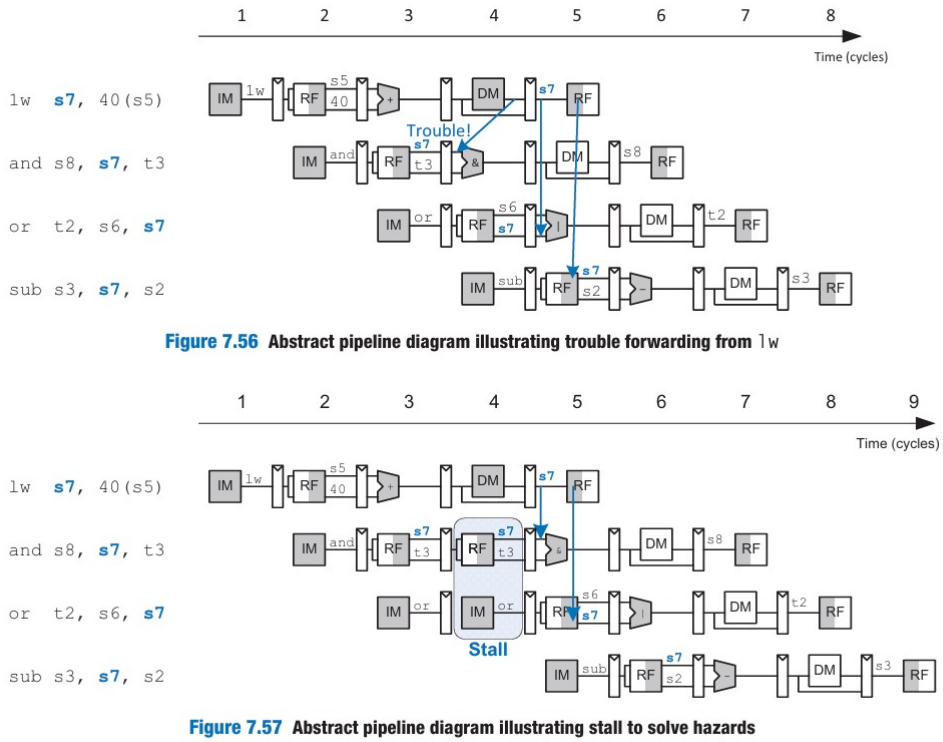


Figure 5: Load-Use Hazard Resolved by a One-Cycle Stall. The bubble propagates through MEM and WB with no side-effects.

### (c) Control Hazards - Branch Flush

A **control hazard** occurs because the pipeline fetches instructions sequentially, but a taken branch redirects the PC to a non-sequential target. Instructions fetched along the fall-through path must be discarded.

**Branch penalty and decision point.** Harris & Harris show that the misprediction penalty depends on when the branch decision is resolved [1]:

- **Memory stage** resolution ⇒ **3-cycle** penalty (three speculative instructions must be flushed).
- **Decode stage** resolution with a dedicated equality comparator ⇒ **1-cycle** penalty.

In our implementation the branch condition (`BranchE` AND `ZeroE`) is evaluated in the **Execute stage**, yielding a **2-cycle penalty**: one instruction in Decode and one in Fetch must be discarded when `PCSrcE` is asserted.

**Solution - Flushing (Squashing):** When `PCSrcE` is asserted, the flush signals in Figure 7 are activated:

- `FlushD = PCSrcE` clears the IF/ID register, discarding the instruction in Decode.
- `FlushE = lwStall | PCSrcE` clears the ID/EX register, handling both load-use bubbles and branch flushes with a single signal.

The PC is updated to the branch target and fetch resumes from the correct path.

**Example:**

```
beq  x8, x8, +8    # always TAKEN -- decided in EX (cycle N)
addi x10, x0, 99   # in Decode at cycle N -- FLUSHED
addi x10, x0, 42   # x10 = 42  (branch target, fetched correctly)
```

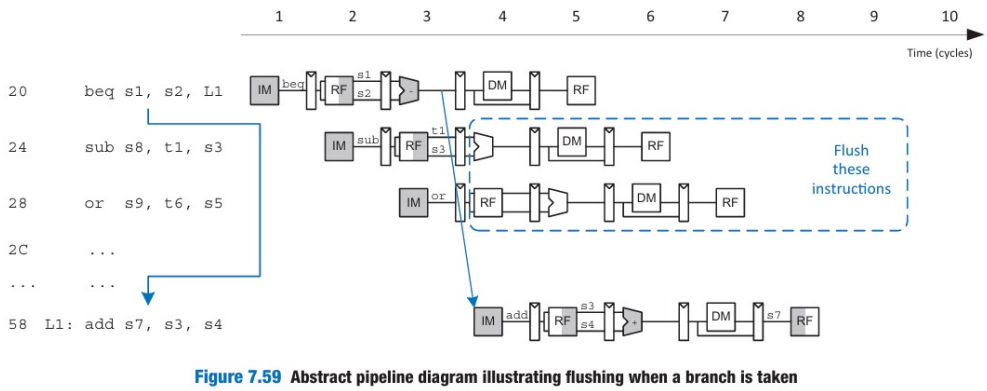


**Figure 7.59** Abstract pipeline diagram illustrating flushing when a branch is taken

Figure 6: Control Hazard: Branch Decided in EX Stage. `FlushD` and `FlushE` discard the speculatively fetched instruction and redirect the PC. Penalty is 2 cycles (vs. 3 for MEM-stage and 1 for ID-stage resolution [1]).

**Forward to solve data hazards when possible**[3]**:**

if $((Rs1E == RdM)\ \&\ RegWriteM)\ \&\ (Rs1E\ !=\ 0)$ then
$ForwardAE = 10$
else if $((Rs1E == RdW)\ \&\ RegWriteW)\ \&\ (Rs1E\ !=\ 0)$ then
$ForwardAE = 01$
else $ForwardAE = 00$

**Stall when a load hazard occurs:**

$lwStall = ResultSrcE_0\ \&\ ((Rs1D == RdE) \mid (Rs2D == RdE))$
$StallF = lwStall$
$StallD = lwStall$

**Flush when a branch is taken or a load introduces a bubble:**

$FlushD = PCSrcE$
$FlushE = lwStall \mid PCSrcE$

Figure 7: Hazard control equations from Harris & Harris: forwarding conditions for `ForwardAE`, load-use stall logic (`lwStall`, `StallF`, `StallD`), and branch flush signals (`FlushD`, `FlushE`) [1].

## Final Pipelined RISC-V Processor with Hazard Handling

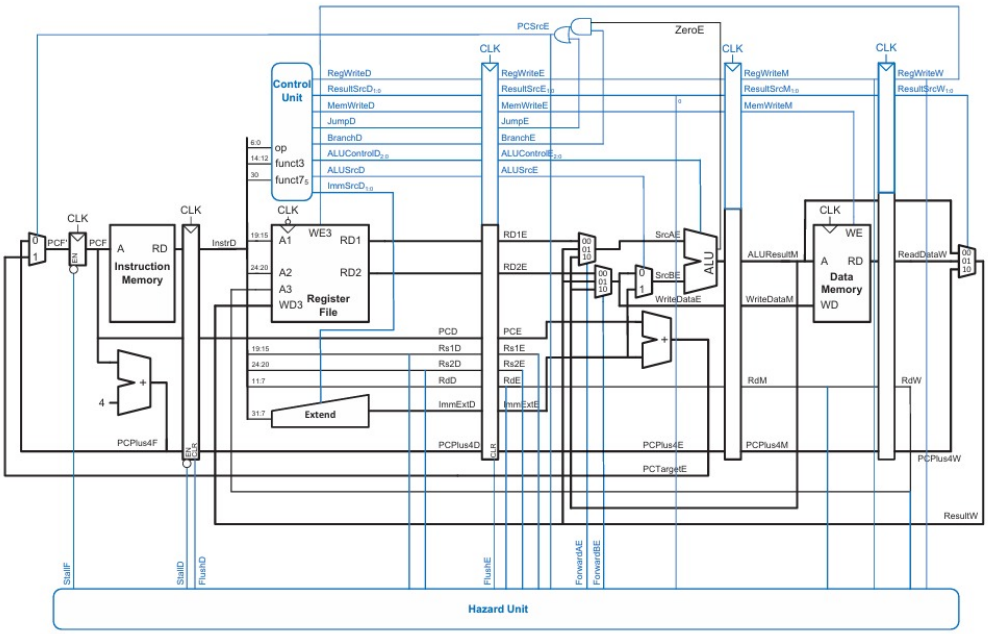


**Figure 7.61** Pipelined processor with full hazard handling

Figure 8: Final Pipelined RISC-V Processor with Forwarding Unit, Hazard Detection Unit, and Branch Flush Logic.

## 1.6 Plots & Screenshots

The following simulation waveforms were captured from the pipelined RISC-V processor testbench. Each plot highlights a specific aspect of pipeline execution and hazard handling.

### Complete Pipeline Execution

Figure 9 shows the full pipeline in operation across multiple clock cycles. All key signals - `InstrF`, `InstrD`, `ImmExtD/E`, `ALUResultE/M`, `ResultW`, `ForwardAE/BE`, `StallF/D`, `FlushE/D` - are displayed. Note that during this segment of execution, no stalls or flushes are triggered (`StallF = 0`, `StallD = 0`, `FlushE = 0`), and instructions flow smoothly through all five pipeline stages.

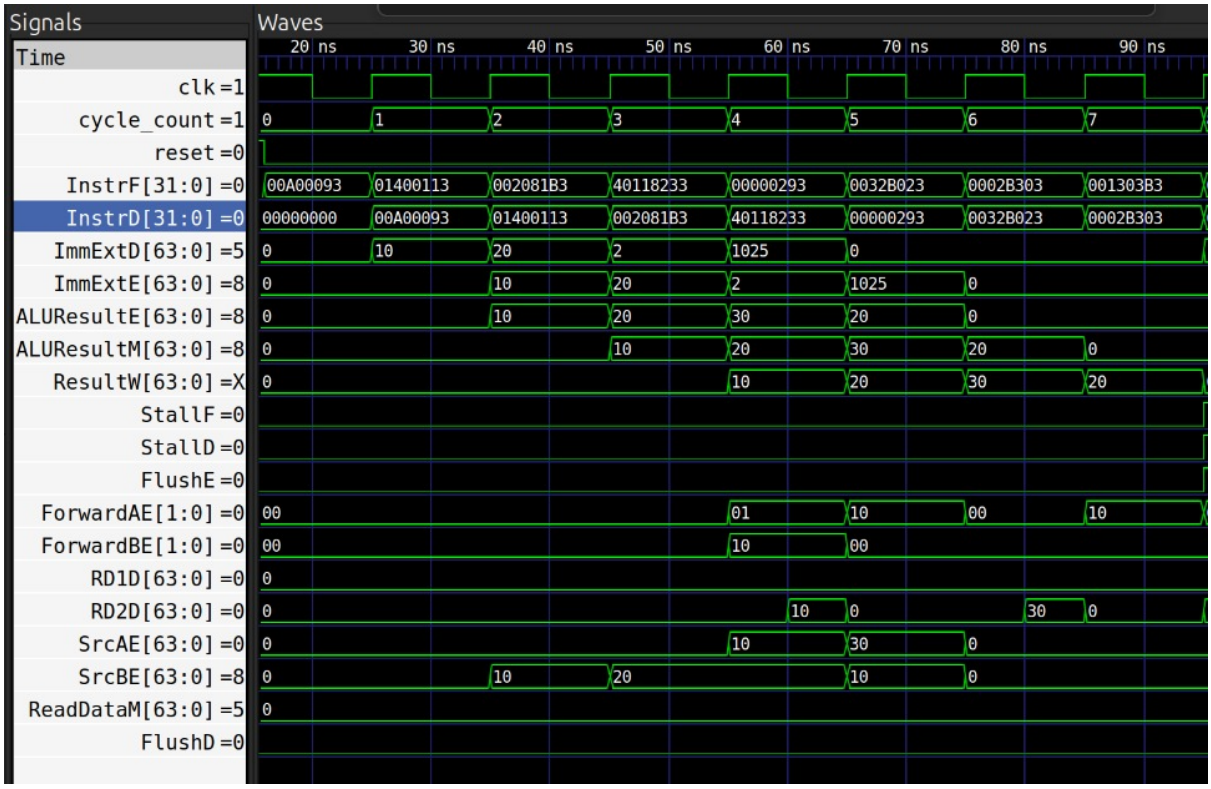


Figure 9: Complete Pipeline Execution - Simulation Waveform

### ALU Forwarding

The following instruction sequence demonstrates ALU forwarding (data bypassing) in action:

```
addi x1, x0, 10    ; x1 = 10
addi x2, x0, 20    ; x2 = 20
add  x3, x1, x2    ; x3 = 30  [MEM->EX fwd x1] [WB->EX fwd x2]
sub  x4, x3, x1    ; x4 = 20  [MEM->EX fwd x3]
```

As shown, in step number 3 there are 2 data forwards that need to happen in order for `add` to be able to perform its task successfully. Our processor takes care of this issue perfectly. In `cycle_count` 4 as highlighted in the wavefrom above, the source register 1 in the execute stage i.e `x1` is equal to the destination address in the write back stage - `x1` (the executing statement here is `addi x1, x0, 10` thus `x1` is a destination register). This dependancy is caught rightly by the hazard unit causing `x1`'s data to be forwarded using a mux. The signal `ForwardAE` is a MuxSelect signal which causes `ResultW` to come here.

A similar analysis can be done for source register 2 as well. As highlighted, in `cycle_4` `Rs2E = RdM = 2` indicating `x2` dependancy. `ForwardBE` is rightly triggered causing the forwarding of `ALUResultM`

Thus, forwarded 20 + forwarded 10 rightly computes 30, as can be seen in `ALUResultE`'s waveform.

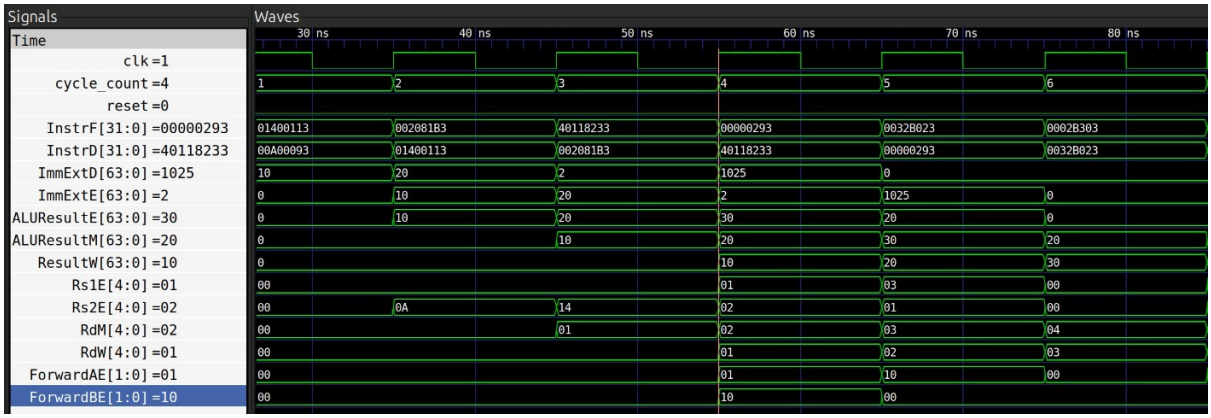


Figure 10: ALU Forwarding (EX/MEM and MEM/WB Bypass) - Simulation Waveform

### Load-Use Stall

```
;  SECTION 2: LOAD-USE STALL
addi x5, x0, 0      ; x5 = 0  (mem base)
sd   x3, 0(x5)      ; mem[0] = 30
ld   x6, 0(x5)      ; x6 = 30
add  x7, x6, x1     ; x7 = 40  [LOAD-USE stall on x6]
```

In this code snippet, a load use hazard takes place. In statement 4 of this subsection, `add` immediately requires `x6`'s data that is loaded in the very previous stage. It is important to note here that `add` requires this data in the execute stage, at which point `ld` is in its memory stage. Forwarding will not fix this issue, since `x6` doesn't get its data until the very end of the memory cycle, while ALU needs it right at the start to proceed with its calculations. Thus, a stall is required. A stall where, `add x7, x6, x1` waits one cycle but `ld x6, 0(x5)` doesn't. This ensures that when `add` comes to execute stage, `ld` is in its write back stage allowing for fruitful forwarding.

In program `cycle_count = 8` as shown, when `add` is in its decode stage and `ld` is in its execute stage, `Rs1D = RdE = 6` indicating `x6`. This dependancy is righly picked by by the hazard unit causing fetch and decode to stall while `ld` can continue to memory stage and fetch `x6`'s data. `InstrF` and `InstrD` as shown in the waveform are held for `cycle_count` 8 and 9 - indicating stall. In `cycle_count = 10`, WB→EX forwarding takes place thus giving `x7` its right value which is 40.

Figure 11 captures the load-use hazard scenario.

Figure 11: Load-Use Hazard - Pipeline Stall Waveform

### Branch Hazard (Flush)

Consider the following code snippet demonstrating a branch hazard:

```
;  SECTION 3: BRANCH
addi x8, x0, 5      ; x8 = 5
addi x9, x0, 0      ; padding
beq  x8, x8, +8     ; always TAKEN → skip x10=99
addi x10, x0, 99    ; SKIPPED (flushed)
addi x10, x0, 42    ; x10 = 42  (branch target)
```

To demonstrate static branch prediction, the above code snippet is used. beq x8, x8, +8 is an always true instruction, thus branch is always taken 2 instrctions after the current. However, this does not happen, the instruction addi x10, x0, 99 (0x06300513) is wrongly taken with it being in decode stage at cycle$_count = 13.Thankfully, inthisstage, BranchEandZeroEare$

Figure 12 shows the control hazard caused by a taken branch.

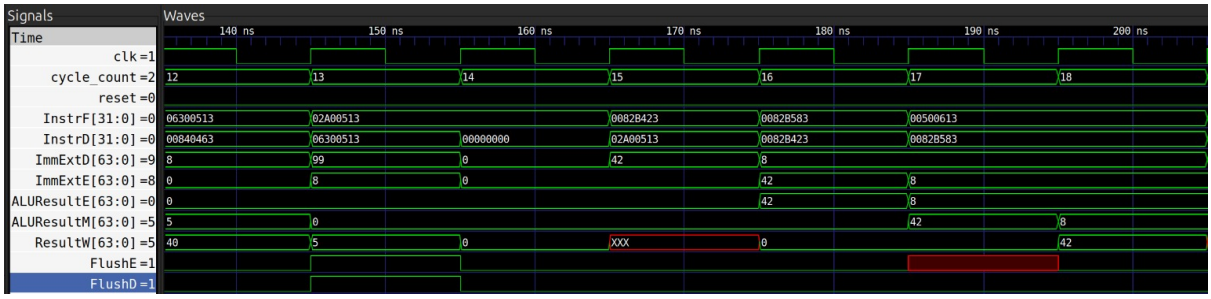


Figure 12: Branch Hazard - Pipeline Flush Waveform

## 1.7 Conclusion

This section presented the design and implementation of a **five-stage pipelined RISC-V processor** (RV64I subset), evolving from the single-cycle sequential design documented earlier in this report. The key contributions and outcomes are summarised below.

1. **Pipelining for Throughput:** By dividing instruction execution into five stages (IF, ID, EX, MEM, WB) and inserting pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) between them, the processor achieves up to a five-fold improvement in throughput over the single-cycle design. In steady state, one instruction completes every clock cycle.

2. **Hazard Detection and Resolution:** Three classes of pipeline hazards were identified and resolved through dedicated hardware units:

   - **Data hazards** - resolved by the *Forwarding Unit*, which bypasses ALU results from the EX/MEM and MEM/WB pipeline registers directly to the ALU inputs, eliminating unnecessary stalls for back-to-back dependent instructions.

   - **Load-use hazards** - resolved by the *Hazard Detection Unit*, which inserts a one-cycle pipeline stall (bubble) when a load instruction is immediately followed by a dependent instruction, after which the data is forwarded normally.

- **Control hazards** - resolved by *flushing* (squashing) incorrectly fetched instructions from the pipeline when a branch is determined to be taken, and redirecting the PC to the correct branch target.

3. **Functional Verification:** Simulation waveforms confirmed the correct operation of the pipeline across all scenarios - normal execution, ALU forwarding, load-use stalling, and branch flushing. The forwarding signals (`ForwardAE`, `ForwardBE`), stall signals (`StallF`, `StallD`), and flush signals (`FlushE`, `FlushD`) were observed to activate precisely when expected, and all computed results matched the expected values.

4. **Design Completeness:** The final pipelined processor integrates the Forwarding Unit, Hazard Detection Unit, and branch flush logic into a cohesive design that correctly executes the supported instruction subset (`add`, `sub`, `addi`, `and`, `or`, `ld`, `sd`, `beq`) while maintaining pipeline efficiency.

The transition from a single-cycle to a pipelined architecture demonstrates the fundamental trade-off in processor design: increased hardware complexity (pipeline registers, forwarding MUXes, hazard detection logic) in exchange for significantly higher instruction throughput and better utilisation of all functional units across every clock cycle.

# Contributions

| Member | Contributions |
|---|---|
| Jayakanth M V | Pipelined Datapath Wrapper, Control Unit, Hazard Unit , Parameterized register |
| Lavanya Patil | Pipelined Datapath Wrapper, Register File, PC, Instruction Memory, Hazard Unit |
| Nihaal Appana | Pipelined Control Path Wrapper, Data Memory, Immediate Generator, Hazard Unit |
| Claude | Testbenches |

Team Contributions

# References

[1] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*. Morgan Kaufmann, 2007.